HANGFUL

Architectural Design Document

CMSI 4072: Senior Project II
Loyola Marymount University

Table of Contents

1. Introduction

This document presents the architectural design of the Hangful iOS application. It describes the system's overall structure, its major components and their interactions, and provides diagrams showing how the parts relate to each other. The architecture is designed to support both the current mock-data MVP and future integration with a real backend API.

2. Architectural Overview

Hangful follows the MVVM (Model-View-ViewModel) architectural pattern, which is the recommended architecture for SwiftUI applications. The application is decomposed into five layers: Models define the data structures; Services provide data access (currently via MockData, designed for replacement with a real API client); ViewModels manage state and business logic using Combine's @Published properties; Views render the user interface declaratively based on ViewModel state; and Components provide reusable UI elements shared across views.

The architecture enforces a strict unidirectional data flow: Views observe ViewModels via @EnvironmentObject injection, ViewModels call Service methods to fetch or mutate data, and Services return updated data which ViewModels publish back to Views. Views never access Services directly, and Services have no knowledge of Views.

3. Major Software Components

3.1 Models Layer

The Models layer consists of a single file (Models.swift) containing all data structures used throughout the application. This includes User, Place (with PlaceCategory enum), Deal (with RewardType enum), Hangout (with HangoutStatus enum), FriendProfile, Friendship, FriendRequest, FriendGroup, FriendAvailability, QuickHangoutCategory, and HangfulError. All models conform to Swift's Codable protocol for future JSON serialization and Identifiable for SwiftUI list rendering. The Hangout model is the central entity, containing references to a Place, an optional Deal, and an array of FriendProfile attendees.

3.2 Services Layer

The Services layer contains MockData.swift, a singleton class that serves as the centralized data source for the entire application. It holds all mock state (users, places, deals, hangouts, friendships, groups) and provides async methods that simulate network latency using Task.sleep. The MockData class is designed as a drop-in replacement target: every method signature (requestOTP, verifyOTP, createHangout, submitProof, etc.) mirrors the API contract of the planned Node.js/Express backend. Switching to a real backend requires creating an APIClient class with identical method signatures and replacing MockData.shared references in the ViewModels.

3.3 ViewModels Layer

The ViewModels layer contains all state management logic in a single file (ViewModels.swift) with six @MainActor ObservableObject classes: AuthViewModel manages the three-state authentication flow (unauthenticated → otpSent → authenticated). HangoutsViewModel manages the hangout feed, the 5-step creation flow, proof submission, and simulated approval. ExploreViewModel manages the place directory with search, category filtering, and deals-only toggle. FriendsViewModel manages the friends list, friend requests, groups, search, and the Friends/Groups tab toggle. CalendarViewModel manages the day selector, week dates computation, and friend availability per day. MapViewModel manages the friend pin data and map region for the Friend Map view.

3.4 Views Layer

The Views layer is organized into six feature directories containing the SwiftUI views. Auth/ contains the app entry point (@main), MainTabView, PhoneInputView, and OTPView. Hangouts/ contains HangoutFeedView (with Quick Hangouts row and HangoutCard), HangoutDetailView, and CreateHangoutView (5-step modal flow). Explore/ contains ExploreView and PlaceDetailView. Friends/ contains FriendsView (with Best Friends row, friend list, groups, request banner), FriendProfileView, and AddFriendView. Calendar/ contains CalendarView, FriendMapView, ProfileView, and BrandRequestSheet. Each view accesses its corresponding ViewModel via @EnvironmentObject.

3.5 Components Layer

The Components layer contains SharedComponents.swift with reusable UI elements used across multiple views: AvatarView generates consistent colored circle avatars from names. StatusChip renders color-coded hangout status badges (confirmed, pending, cancelled, etc.).

WeekDayRow renders the Mon–Sun day selector with availability bars, used in both CalendarView and FriendProfileView. PlaceRow renders a place list item with category icon, name, address, and deal badge. Triangle is a custom Shape used for map pin tails.

4. Component Interactions

4.1 App Initialization Flow

HangfulApp (@main) creates five StateObject ViewModels and injects them into the view hierarchy via environmentObject. The root view switches between PhoneInputView, OTPView, and MainTabView based on AuthViewModel.authState. On authentication, the MainTabView renders four tabs, each with its own NavigationStack and feature views.

4.2 Data Flow Pattern

All data flows through the MockData singleton. ViewModels call MockData methods on .onAppear or user actions, receive updated data, and publish it via @Published properties. Views re-render automatically via SwiftUI's declarative binding. For example: User taps 'Create Hangout' → CreateHangoutView calls HangoutsViewModel.createHangout() → HangoutsViewModel calls MockData.createHangout() → MockData appends to its hangouts array and returns the new Hangout → HangoutsViewModel updates its @Published hangouts array → HangoutFeedView automatically re-renders with the new card.

4.3 Cross-Feature Dependencies

The hangout creation flow depends on ExploreViewModel for the place directory. The FriendProfileView uses CalendarViewModel's WeekDayRow for the availability section. The proof approval flow in HangoutsViewModel triggers a referral unlock by mutating MockData's currentUser.firstHangoutCompleted flag, which ProfileView observes. The MapViewModel reads from MockData's friendships array to generate map pins, ensuring map data stays in sync with the friends list.

5. File Structure

The application consists of 10 Swift files organized into 5 directories:

File	Contents
Models/Models.swift	All data structures: User, Place, Deal, Hangout, FriendProfile, Friendship, FriendRequest, FriendGroup, FriendAvailability, QuickHangoutCategory, HangfulError, enums
Services/MockData.swift	Singleton mock data source with all state and async methods simulating API calls
ViewModels/ViewModels.swift	AuthViewModel, HangoutsViewModel, ExploreViewModel, FriendsViewModel, CalendarViewModel, MapViewModel
Components/SharedComponents.swift	AvatarView, StatusChip, WeekDayRow, PlaceRow, Triangle
Views/Auth/AppAndAuth.swift	@main entry, MainTabView, PhoneInputView, OTPView
Views/Hangouts/HangoutFeedView.swift	Home tab: Quick Hangouts row, HangoutCard, feed sections
Views/Hangouts/HangoutViews.swift	HangoutDetailView (proof, redemption, share), CreateHangoutView (5-step)
Views/Explore/ExploreView.swift	Place browser with filters, PlaceDetailView with deal info
Views/Friends/FriendsViews.swift	FriendsView, FriendProfileView, AddFriendView
Views/Calendar/CalMapProfile.swift	CalendarView, FriendMapView, ProfileView, BrandRequestSheet

6. Future Architecture (Backend Integration)

When the backend is implemented, the architecture transitions from a two-layer system (iOS + MockData) to a three-tier client-server architecture (iOS App → Node.js/Express API → PostgreSQL on Supabase). The MockData singleton is replaced with an APIClient class that makes HTTPS REST calls with JSON payloads and JWT Bearer token authentication. The ViewModel layer and View layer require no structural changes — only the data source reference changes from MockData.shared to APIClient.shared. The backend API provides 15+ endpoints covering authentication (OTP via Twilio), hangout CRUD, proof submission, friend management, availability, and admin proof approval.